

visualize2.py

```
1 # import numpy as np
2 # import torch
3 # import torch.nn.functional as F
4 # import matplotlib
5 # matplotlib.use("Agg")
6 # import matplotlib.pyplot as plt
7
8 # from config2 import (DEVICE, FIGURES_DIR, NUM_CLASSES, CLASS_NAMES,
9 #                       IN_CHANNELS, IMAGE_SIZE, version_model_path)
10 # from model2 import SamVGG16Ex2
11 # from dataset2 import build_transforms
12 # from PIL import Image
13
14
15 # def _load_best(best_version, dropout=0.5):
16 #     ckpt = torch.load(version_model_path(best_version),
17 #                       map_location=DEVICE, weights_only=False)
18 #     model = SamVGG16Ex2(NUM_CLASSES, IN_CHANNELS, dropout=ckpt.get("dropout", 0.5)).to(DEVICE)
19 #     model.load_state_dict(ckpt["model_state"]); model.eval()
20 #     return model
21
22
23 # def _one_image_per_class(val_df, eval_tf):
24 #     """Return list of (class_idx, tensor[1,1,H,W], display_np) - one per class."""
25 #     out = []
26 #     for c in range(NUM_CLASSES):
27 #         sub = val_df[val_df["label"] == c]
28 #         if len(sub) == 0:
29 #             continue
30 #         fp = sub.iloc[0]["filepath"]
31 #         pil = Image.open(fp).convert("L")
32 #         t = eval_tf(pil).unsqueeze(0)
33 #         out.append((c, t, np.asarray(pil.resize((IMAGE_SIZE, IMAGE_SIZE)))))
34 #     return out
35
36
37 # def feature_maps(model, samples):
38 #     """Grid: rows = classes, cols = mean activation after each conv block."""
39 #     # find block boundaries (after each MaxPool)
40 #     block_idx = [i for i, m in enumerate(model.features)
41 #                  if isinstance(m, torch.nn.MaxPool2d)]
42 #     fig, axes = plt.subplots(len(samples), len(block_idx) + 1,
43 #                             figsize=(2.2 * (len(block_idx) + 1), 2.2 * len(samples)))
44 #     if len(samples) == 1:
45 #         axes = axes[None, :]
46 #     for r, (c, t, disp) in enumerate(samples):
47 #         axes[r, 0].imshow(disp, cmap="gray"); axes[r, 0].axis("off")
48 #         axes[r, 0].set_ylabel(CLASS_NAMES[c])
49 #         if r == 0: axes[r, 0].set_title("input")
50 #         x = t.to(DEVICE); col = 1
51 #         with torch.no_grad():
52 #             for i, layer in enumerate(model.features):
53 #                 x = layer(x)
54 #                 if i in block_idx:
55 #                     fmap = x.mean(1).squeeze(0).cpu().numpy() # mean over channels
56 #                     axes[r, col].imshow(fmap, cmap="viridis"); axes[r, col].axis("off")
57 #                     if r == 0: axes[r, col].set_title(f"block{col}")
58 #                     col += 1
59 #     plt.tight_layout(); p = FIGURES_DIR / "feature_maps_per_class.png"
60 #     plt.savefig(p, dpi=150, bbox_inches="tight"); plt.close(); print(f" ✓ {p}")
61
62
63 # def saliency_maps(model, samples):
64 #     fig, axes = plt.subplots(2, len(samples), figsize=(3 * len(samples), 6))
65 #     for j, (c, t, disp) in enumerate(samples):
66 #         x = t.clone().to(DEVICE).requires_grad_(True)
67 #         logits = model(x)
68 #         score = logits[0, c]
69 #         model.zero_grad()
70 #         score.backward()
```

```

71 #         sal = x.grad.abs().squeeze().cpu().numpy() # |do/dx|, 1 channel
72 #         axes[0, j].imshow(dis, cmap="gray"); axes[0, j].axis("off")
73 #         axes[0, j].set_title(CLASS_NAMES[c])
74 #         axes[1, j].imshow(sal, cmap="hot"); axes[1, j].axis("off")
75 #         if j == 0:
76 #             axes[0, j].set_ylabel("input"); axes[1, j].set_ylabel("saliency")
77 #         plt.tight_layout(); p = FIGURES_DIR / "saliency_maps.png"
78 #         plt.savefig(p, dpi=150, bbox_inches="tight"); plt.close(); print(f" ✓ {p}")
79
80
81 # def grad_cam(model, samples):
82 #     # last conv layer in features
83 #     last_conv = max(i for i, m in enumerate(model.features)
84 #                     if isinstance(m, torch.nn.Conv2d))
85 #     acts, grads = {}, {}
86 #     def fwd_hook(_, __, out): acts["v"] = out.detach()
87 #     def bwd_hook(_, gin, gout): grads["v"] = gout[0].detach()
88 #     h1 = model.features[last_conv].register_forward_hook(fwd_hook)
89 #     h2 = model.features[last_conv].register_full_backward_hook(bwd_hook)
90
91 #     fig, axes = plt.subplots(2, len(samples), figsize=(3 * len(samples), 6))
92 #     for j, (c, t, disp) in enumerate(samples):
93 #         x = t.clone().to(DEVICE)
94 #         logits = model(x); model.zero_grad(); logits[0, c].backward()
95 #         a, g = acts["v"][0], grads["v"][0] # [C,h,w]
96 #         weights = g.mean(dim=(1, 2)) # GAP over spatial
97 #         cam = F.relu((weights[:, None, None] * a).sum(0))
98 #         cam = cam / (cam.max() + 1e-8)
99 #         cam = F.interpolate(cam[None, None], size=(IMAGE_SIZE, IMAGE_SIZE),
100 #                             mode="bilinear", align_corners=False)[0, 0].cpu().numpy()
101 #         axes[0, j].imshow(dis, cmap="gray"); axes[0, j].axis("off")
102 #         axes[0, j].set_title(CLASS_NAMES[c])
103 #         axes[1, j].imshow(dis, cmap="gray")
104 #         axes[1, j].imshow(cam, cmap="jet", alpha=0.5); axes[1, j].axis("off")
105 #     h1.remove(); h2.remove()
106 #     plt.tight_layout(); p = FIGURES_DIR / "grad_cam.png"
107 #     plt.savefig(p, dpi=150, bbox_inches="tight"); plt.close(); print(f" ✓ {p}")
108
109
110 # def run_all_visualizations(best_version, val_df):
111 #     print("\n — CNN visualization on best model —")
112 #     _, eval_tf = build_transforms("geometric") # clean transform only
113 #     model = _load_best(best_version)
114 #     samples = _one_image_per_class(val_df, eval_tf)
115 #     feature_maps(model, samples)
116 #     saliency_maps(model, samples)
117 #     grad_cam(model, samples)
118 import numpy as np
119 import torch
120 import torch.nn.functional as F
121 import matplotlib
122 matplotlib.use("Agg")
123 import matplotlib.pyplot as plt
124
125 from config2 import (DEVICE, FIGURES_DIR, NUM_CLASSES, CLASS_NAMES,
126                     IN_CHANNELS, IMAGE_SIZE, version_model_path)
127 from model2 import SamVGG16Ex2
128 from dataset2 import build_transforms
129 from PIL import Image
130
131 def _load_best(best_version, dropout=0.5):
132     ckpt = torch.load(version_model_path(best_version),
133                       map_location=DEVICE, weights_only=False)
134     model = SamVGG16Ex2(NUM_CLASSES, IN_CHANNELS, dropout=ckpt.get("dropout", 0.5)).to(DEVICE)
135     model.load_state_dict(ckpt["model_state"])
136     model.eval()
137     return model
138
139 def _three_random_cases_per_class(val_df, eval_tf, target_classes=[0, 1, 2, 3], num_cases=3):
140     out = []
141     for c in target_classes:
142         sub = val_df[val_df["label"] == c]

```

```

143     if len(sub) == 0:
144         continue
145     sampled = sub.sample(n=min(len(sub), num_cases))
146     for _, row in sampled.iterrows():
147         fp = row["filepath"]
148         pil = Image.open(fp).convert("L")
149         t = eval_tf(pil).unsqueeze(0)
150         disp = np.asarray(pil.resize((IMAGE_SIZE, IMAGE_SIZE)))
151         out.append((c, t, disp))
152     return out
153
154 def combined_visualizations(model, samples):
155     last_conv = max(i for i, m in enumerate(model.features)
156                    if isinstance(m, torch.nn.Conv2d))
157     acts, grads = {}, {}
158
159     def fwd_hook(_, __, out): acts["v"] = out.detach()
160     def bwd_hook(_, gin, gout): grads["v"] = gout[0].detach()
161
162     h1 = model.features[last_conv].register_forward_hook(fwd_hook)
163     h2 = model.features[last_conv].register_full_backward_hook(bwd_hook)
164
165     fig, axes = plt.subplots(len(samples), 4, figsize=(12, 3 * len(samples)))
166
167     for r, (c, t, disp) in enumerate(samples):
168         x = t.clone().to(DEVICE).requires_grad_(True)
169         logits = model(x)
170         score = logits[0, c]
171         model.zero_grad()
172         score.backward()
173
174         axes[r, 0].imshow(disp, cmap="gray")
175         axes[r, 0].axis("off")
176         axes[r, 0].set_title(f"Class: {CLASS_NAMES[c]}")
177
178         fmap = acts["v"][0].mean(0).cpu().numpy()
179         axes[r, 1].imshow(fmap, cmap="viridis")
180         axes[r, 1].axis("off")
181         if r == 0: axes[r, 1].set_title("Last Feature Map")
182
183         sal = x.grad.abs().squeeze().cpu().numpy()
184         axes[r, 2].imshow(sal, cmap="hot")
185         axes[r, 2].axis("off")
186         if r == 0: axes[r, 2].set_title("Saliency")
187
188         a, g = acts["v"][0], grads["v"][0]
189         weights = g.mean(dim=(1, 2))
190         cam = F.relu((weights[:, None, None] * a).sum(0))
191         cam = cam / (cam.max() + 1e-8)
192         cam = F.interpolate(cam[None, None], size=(IMAGE_SIZE, IMAGE_SIZE),
193                            mode="bilinear", align_corners=False)[0, 0].cpu().numpy()
194
195         axes[r, 3].imshow(disp, cmap="gray")
196         axes[r, 3].imshow(cam, cmap="jet", alpha=0.5)
197         axes[r, 3].axis("off")
198         if r == 0: axes[r, 3].set_title("Grad-CAM")
199
200     h1.remove()
201     h2.remove()
202     plt.tight_layout()
203     p = FIGURES_DIR / "combined_visualizations.png"
204     plt.savefig(p, dpi=150, bbox_inches="tight")
205     plt.close()
206     print(f" ✓ {p}")
207
208 def run_all_visualizations(best_version, val_df):
209     print("\n — CNN visualization on best model —")
210     _, eval_tf = build_transforms("geometric")
211     model = _load_best(best_version)
212     target_classes = list(range(min(4, NUM_CLASSES)))
213     samples = _three_random_cases_per_class(val_df, eval_tf, target_classes, num_cases=3)
214     combined_visualizations(model, samples)

```